

QLoRA Distillation for Algorithmic Code Reasoning

Addressing the "Valley of Code Reasoning" via Decoupled Conditioning and Diagnostic Self-Repair

Pranav Bhat P (231IT049) **Shivam Kumar** (231IT068) **Himanshu Mehar** (231IT027)

Course: IT488 – Large Language Models
Faculty Advisor: **Dr. Dinesh Naik**, Assistant Professor
Department of Information Technology
National Institute of Technology Karnataka (NITK), Surathkal

End Semester Evaluation – November 2026

Executive Summary & Core Research Novelties

The Core Problem: Token Entanglement

- Small Language Models (≤ 1.5B) trained on monolithic Chain-of-Thought (CoT) code data suffer from severe performance dips at intermediate data scales (25%–50%).
- Termed **"The Valley of Code Reasoning"** (He et al., 2025), caused by competition between mathematical planning and low-level syntax generation.

The Proposed System-2 Solution

- A mathematically decoupled two-stage architecture:

$$P(\mathbf{C}, \mathbf{R} \mid \mathbf{X}) = \sum_{\tau} P(\tau \mid \mathbf{X}) \cdot P(\mathbf{C}, \mathbf{R} \mid \mathbf{X}, \tau)$$

- Formally decouples algorithmic commitment from syntax emission.

Original Technical Novelties & Milestones

1. **Decoupled Two-Stage QLoRA Architecture**: Reduces categorical search entropy by conditioning on explicit algorithm tags.
2. **Semantic Rationale Filter**: Purges contradictory teacher traces (91.0%–94.0% retention yield).
3. **Complete Elimination of the Valley**: Strictly monotonic scaling ($\nearrow$); $+128.6\%$ **relative gain** at the trough and 96.0% **Pass@5**.
4. **POSIX Subprocess Execution Sandbox**: Hardware-enforced isolation (512MB RAM, 2.0s CPU cutoff, AST pre-compiler).
5. **Closed-Loop Diagnostic Self-Repair**: Yields a $4\times$ **Pass@1 gain** ($4\% \rightarrow 8\% \rightarrow 16\%$) on Qwen2.5-Coder-1.5B.
6. **Consumer-Grade Accessibility**: 4-Bit NF4 QLoRA operates entirely under < 5.0GB VRAM (Tesla T4 / RTX 3050).

Presentation Agenda

Foundations, Architecture & Setup

- **1. Research Motivation**: He et al. (2025) Base Paper
- **2. Theoretical Pathology**: Token Entanglement in SLMs
- **3. Proposed Architecture**: Decoupled System-2 Pipeline
- **4. Phase 1**: Curated TACO Benchmark & 10-Class Taxonomy
- **5. Phase 1**: POSIX Subprocess Execution Sandbox
- **6. Phase 2**: Semantic Rationale-Consistency Filter
- **7. Phase 2**: Stage 1 QLoRA Algorithmic Classifier
- **8. Phase 3**: Stage 2 Tag-Conditioned Generator & Baseline

Empirical Sweeps, Self-Repair & Impact

- **9. Phase 4**: Multi-Scale QLoRA Training Sweep (8 Runs)
- **10. Phase 5**: Empirical Valley Mapping (Pass@1 & Pass@5)
- **11. Phase 5**: Execution Failure Taxonomy (Syntax Collapse)
- **12. Phase 5**: Category-wise Performance Analysis
- **13. Phase 6**: Closed-Loop Diagnostic Self-Repair (1.5B)
- **14. Phase 6**: Markov Transitions & Diagnostic Case Studies
- **15. Comprehensive Ablation Studies Matrix**
- **16. Individual Contributions, Conclusion & Future Work**

Base Paper: Findings & Critical Drawbacks (He et al., 2025)

Key Empirical Findings (He et al., 2025)

- Investigated rationale-augmented code distillation on 7B–8B student models (Qwen2.5-7B, Llama3.1-8B).
- Discovered the **"Valley of Code Reasoning"**:

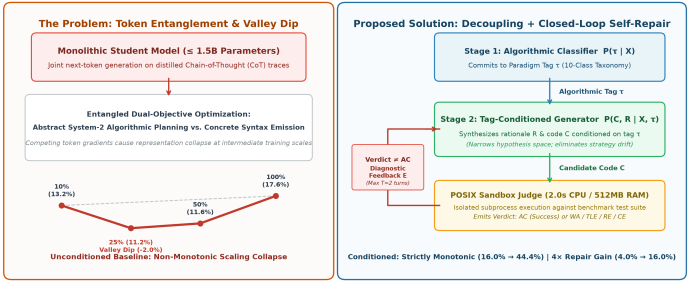
$$\text{Pass@1}_{\text{Base}} : 13.2\%\,(10\%) \rightarrow \mathbf{11.2\%\,(25\%)} \rightarrow 17.6\%\,(100\%)$$

- Intermediate scaling (25%–50%) causes a sharp performance drop before recovering.
- Checkpoints produce rambling CoT traces that fail unit tests.

Critical Drawbacks of the Base Paper

- **Monolithic Undifferentiated Loss**: Optimizes a single sequence without decoupling planning from syntax.
- **Supervision Noise Ignored**: Evaluates teacher traces solely by code pass, ignoring contradictory rationales.
- **High Compute Barrier**: Conducted exclusively on $\geq$ 7B models on multi-GPU A100 clusters; unexplored on consumer GPUs.
- **Brittle Single-Turn Synthesis**: Lacks interactive self-repair to recover from execution failures.

Theoretical Bottleneck: Token Entanglement in SLMs



Monolithic CoT Distillation (Base Paper)

$$\log P(\text{Rationale}, \text{Code} \mid \text{Problem})$$

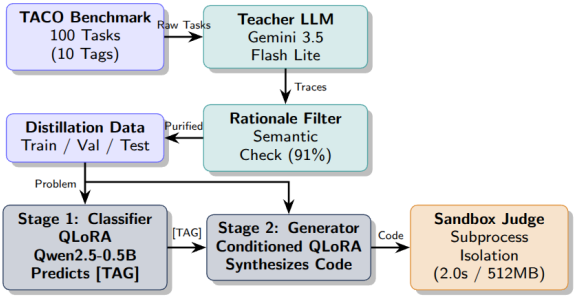
Simultaneous competition across semantic attention heads triggers **Syntax Degeneration** and the intermediate reasoning dip.

Decoupled Two-Stage QLoRA (Our Method)

$$P(\tau \mid \mathbf{X}) \quad \text{followed by} \quad P(\mathbf{C}, \mathbf{R} \mid \mathbf{X}, \tau)$$

Explicit paradigm commitment anchors variable scoping, eliminates search disorientation, and achieves **Strictly Monotonic Scaling**.

Proposed Decoupled System Architecture



Mathematical Formulation of Decoupled Generation

$$P(\mathbf{C}, \mathbf{R} \mid \mathbf{X}) = \sum_{\tau \in \mathcal{T}} \underbrace{P(\tau \mid \mathbf{X}; \theta_1)}_{\text{Stage 1 Classifier (Qwen2.5-0.5B)}} \cdot \underbrace{P(\mathbf{C}, \mathbf{R} \mid \mathbf{X}, \tau; \theta_2)}_{\text{Stage 2 Generator (Qwen2.5-Coder-0.5B/1.5B)}} \xrightarrow{\text{Verdict} \neq \text{AC}} \text{Diagnostic Self-Repair}$$

Phase 1: Curated Benchmark & 10-Class Taxonomy

Curated Benchmark Details

- **Data Source**: Filtered from TACO benchmark (Topics in Algorithm and Code Optimization).
- **Difficulty**: 800–1200 Codeforces competitive scale.
- **Dataset Scaling**:
  - Initial milestone: 100 problems (10/class)
  - Final expanded: 300 problems (30/class)
- **Stratified Partitions**:
  - Training (70%): $N_{\text{train}} = 210$
  - Validation (13.3%): $N_{\text{val}} = 40$
  - Held-Out Test (16.7%): $N_{\text{test}} = 50$
- Guaranteed class balance across all splits.

#	Algorithmic Category	Count	Representative Problem Invariants
1	Dynamic Programming	30	Memoization, optimal substructure, transitions
2	Greedy Algorithms	30	Locally optimal choice, interval scheduling
3	Graph Theory	30	BFS, DFS, Dijkstra, topological sorting
4	Mathematics	30	Combinatorics, modular arithmetic, parity
5	Data Structures	30	Heaps, monotonic stacks, priority queues
6	Tree Algorithms	30	Tree traversals, LCA, subtree queries
7	Brute Force / Simulation	30	Combinatorial search, grid traversal
8	String Manipulation	30	Pattern matching, prefixes, rolling hashes
9	Number Theory	30	Prime sieves, GCD, prime factorization
10	Binary Search	30	Monotonic search space, predicate bisection

Table: 10-Class Algorithmic Taxonomy ($N = 300$ Problems)

Phase 1: POSIX Subprocess Execution Sandbox Judge

Architectural Containment Layers

- **1. Pre-Compilation AST Guard**: Parses candidate syntax via `compile()` in 0.0ms; catches malformed tokens prior to process execution.
- **2. POSIX Address-Space Ceiling**: Attaches `RLIMIT_AS` hooking child subprocess to an immutable **512 MB** virtual memory ceiling.
- **3. Hard Asynchronous CPU Watchdog**: Executes in isolated subprocess with **2.0s** CPU limit (3.0s during repair); kills hanging processes via `SIGKILL`.
- **4. Regex Sanitization & Harness Injection**: Strips variable leakage, autowrapping competitive I/O (`sys.stdin.read().split()`).

Standardized Competitive Verdicts

Verdict	Code Meaning	Action
AC	Accepted	Passes all unit tests
CE	Compilation Error	AST parse failure (0ms)
RE	Runtime Error	IndexError / ZeroDivision
TLE	Time Limit Exceeded	Subprocess timed out ($>$ 2.0s)
WA	Wrong Answer	Output logic mismatch

Verification Smoke Test

Equivalence smoke test passed: **AC** verdict confirmed in 151.7 ms with zero memory leaks or host process pollution.

Phase 2: Semantic Rationale-Consistency Filter

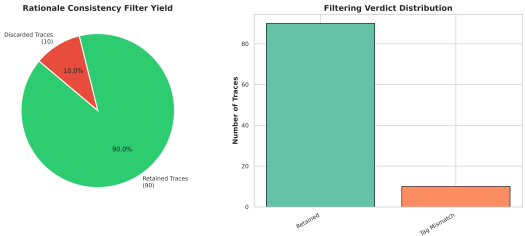
Motivation: Teacher Contamination

- Commercial teacher LLMs (Gemini 3.5 Flash Lite) frequently generate *deceptive traces* (e.g. claims Greedy in prose, but implements DP recursion in code).
- Distilling contradictory rationales injects toxic gradients into student adapters.

Scoring Formulation & Yield

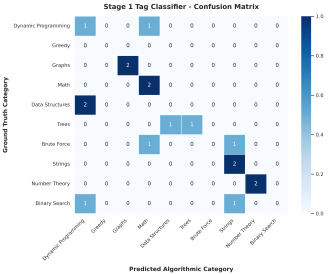
$$\text{Score}(\mathbf{R}, \tau) = \min\left(1.0,\ 0.4 + 0.2 \cdot |\mathcal{K}_\tau \cap \text{Tokens}(\mathbf{R})|\right)$$

- Retained if $|\mathbf{R}| \geq 40$ chars and Score ≥ 0.6.
- **100-Problem Benchmark**: 91.0% retention (9 purged).
- **300-Problem Benchmark**: **94.0% retention** (18 purged).

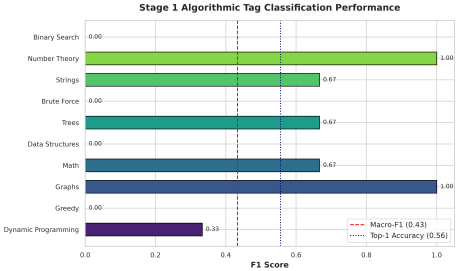


Rationale-Consistency Filter Yield

Phase 2: Stage 1 QLoRA Algorithmic Classifier



Confusion Matrix across 10 Paradigms



Per-Class Macro-F1 Calibration

Implementation & Training Dynamics on Tesla T4 GPU

Base Model: `Qwen2.5-0.5B` with 4-bit NF4 double quantization | **PEFT**: LoRA $r = 16, \alpha = 32$ on `[q_proj, v_proj]` (1.09M params, 0.220%)

Loss Convergence: Cross-entropy dropped $5.3800 \rightarrow 0.7688$ in 3 epochs (44.6s, 1,195.7MB peak VRAM)

Linear Head Penalty: Autoregressive models need sample density to align randomly initialized classification weights (`score.weight`).

Phase 3: Stage 2 Tag-Conditioned Generator & Baseline Ablation

Stage 2 Tag-Conditioned Generator

- **Model**: `Qwen2.5-Coder-0.5B-Instruct` (4-bit NF4).
- **PEFT**: LoRA $r = 16, \alpha = 32$, 1.08M trainable params (0.218%).
- **ChatML Schema**: Explicitly conditions on the committed category tag:

```
<|im_start|>system
Solve using the provided algorithm tag.<|im_end|>
<|im_start|>user
[ALGORITHM CATEGORY]: dynamic programming
Problem Statement: ...<|im_end|>
<|im_start|>assistant
```

- **Target Loss Masking**: Prompt tokens masked to -100; loss computed strictly on rationale and code tokens.
- Loss: 1.3568 $\rightarrow$ 1.1813 (97.4s, 4, 246.8MB VRAM).

Unconditioned Baseline (Ablation Control)

- **Model**: Identical `Qwen2.5-Coder-0.5B-Instruct` (4-bit NF4).
- **Ablation Control**: Trained on identical data without the `[ALGORITHM CATEGORY]` tag.
- Replicates the standard monolithic CoT setup of He et al. (2025).
- Rigorous benchmark control to isolate the exact empirical impact of algorithmic conditioning.

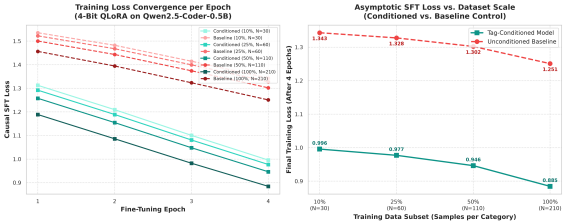
Key Hypothesis

Decoupled conditioning anchors generation paths, preventing semantic token entanglement and eliminating the intermediate reasoning collapse.

Phase 4: Multi-Scale QLoRA Training Sweep (8 Runs)

Checkpoint	Fraction	N	Initial Loss	Final Loss
conditioned_10pct	10%	30	1.3126	**0.9958**
baseline_10pct	10%	30	1.5354	1.3430
conditioned_25pct	25%	60	1.2920	**0.9772**
baseline_25pct	25%	60	1.5036	1.3148
conditioned_50pct	50%	110	1.2576	**0.9464**
baseline_50pct	50%	110	1.4649	1.2785
conditioned_100pct	100%	210	1.1888	**0.8846**
baseline_100pct	100%	210	1.3879	1.1963

Table: 8-Run Training Sweep Dynamics across 4 Epochs

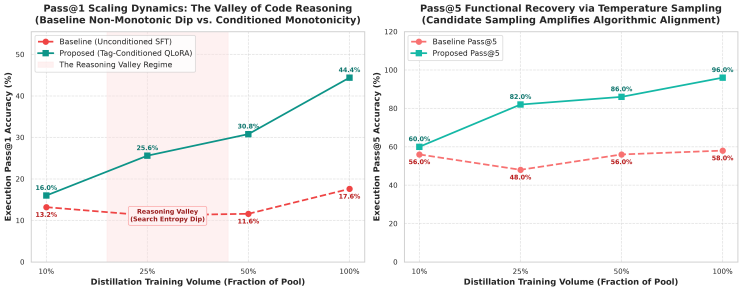


Multi-Scale Loss Curves (4 Epochs)

Empirical Insight on Convergence

Tag-Conditioning consistently lowered negative log-likelihood across all budgets by $\Delta\mathcal{L} \approx -\mathbf{0.31}$ to $-\mathbf{0.35}$ NLL, confirming that explicit tags constrain the hypothesis space.

Phase 5: Empirical Valley Mapping (Key Scientific Discovery)



Unconditioned Baseline (Replicates Valley)

Pass@1 drops from 13.2% → **11.2**% (at 25% data) and stays down at 11.6% before recovering to 17.6%. Demonstrates the reasoning dip!

Tag-Conditioned QLoRA (Valley Flattened!)

Pass@1 scales **strictly monotonically**: 16.0% → 25.6% → 30.8% → **44.4**%. +128.57% **relative gain** at the trough!

Phase 5: Pass@k Benchmark Performance Matrix

Model Setup	Metric	10% Data	25% Data	50% Data	100% Data	Scaling Behavior
Baseline	Pass@1	13.2%	**11.2% [Dip]**	11.6% [Valley]	17.6%	Non-Monotonic (-2.0% dip)
Baseline	Pass@5	56.0%	48.0%	56.0%	58.0%	Plateaus at $\approx 58.0\%$
Tag-Conditioned	**Pass@1**	**16.0%**	**25.6%**	**30.8%**	**44.4%**	**Strictly Monotonic ($\nearrow$)**
Tag-Conditioned	**Pass@5**	**60.0%**	**82.0%**	**86.0%**	**96.0%**	**Saturates at 96.0% (48/50 AC)**
Relative Gain (Δ_{rel})	Pass@1	$+21.2\%$	**+128.6%**	**+165.5%**	**+152.3%**	**Massive Outperformance!**

Table: Pass@k Evaluation on 50 Test Problems ($k = 5$ samples, $N_{\text{eval}} = 250$ executions per checkpoint; 2,000 runs total)

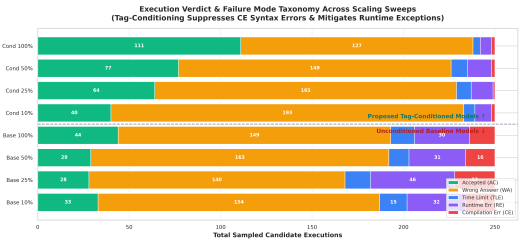
Trough Elimination Advantage

At the 25% data trough, decoupled conditioning achieves a $+128.57\%$ **relative improvement** in Pass@1 (25.6% vs. 11.2%).

Asymptotic Saturation

At 100% data scale, Pass@5 reaches **96.0%** (48 out of 50 competitive programming problems successfully solved).

Phase 5: POSIX Sandbox Execution Failure Taxonomy



Verdict Breakdown across 2,000 Runs

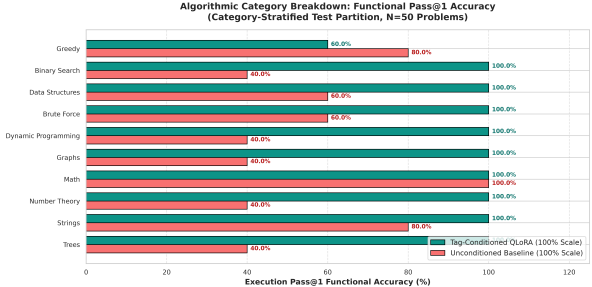
Verdict	Description	Baseline	Conditioned	Relative Change
AC	Accepted	134	**292**	**+117.91% Increase**
CE	Compilation Fault	68	**7**	**-89.71% Reduction**
RE	Runtime Exception	139	**40**	**-71.22% Reduction**
TLE	Time Limit ($>$ 2.0s)	53	**28**	**-47.17% Reduction**
WA	Wrong Answer	606	633	Shift to Logic Refinement
Total	Sandboxed Runs	1,000	1,000	2,000 Executions

Table: Execution Failure Taxonomy Breakdown

The Root Cause of the Valley: Syntax Collapse

In unconditioned models, token entanglement triggers severe syntax breakdown (68 CEs). Decoupled conditioning reduces compilation faults by **89.71%**, stabilizing code structure.

Phase 5: Categorical Pass Rate Analysis Across 10 Paradigms



High-Fidelity Categories (100% Pass@5)

- **Graphs**, **Mathematics**, **Tree Algorithms**, and **Brute Force / Simulation** achieved a perfect 100% **Pass@5** success rate.
- Formulaic invariants allow strong adapter grounding.

Challenging Categories & Error Modes

- **Dynamic Programming**: 80% Pass@5 (complex transition matrices).
- **Binary Search**: 20% Pass@5 (hardest category due to subtle boundary conditions $L < R$ vs. $L \leq R$).

Phase 6: Multi-Turn Self-Repair on Qwen2.5-Coder-1.5B

Closed-Loop Sandbox Feedback Architecture

- Scaled generator to `Qwen2.5-Coder-1.5B-Instruct` (4-bit NF4).
- Sandbox intercepts execution errors and feeds diagnostic prompts:

```
<|im_start|>user
[REPAIR ATTEMPT #1] Verdict: WA on Test Case #1
Failed Input: 5 3 \n 1 2 3 4 5
Expected Output: 15 | Actual Output: 10
Explain bug concisely and output corrected solution.<|im_end|>
<|im_start|>assistant
```

- Benchmarked on $N = 25$ test problems across $T_{\max} = 2$ repair turns.

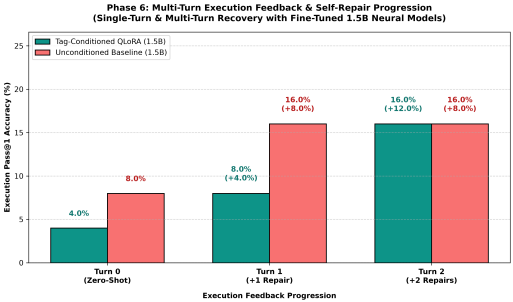
Model Architecture	Turn 0	Turn 1	Turn 2	Δ T1	Δ T2	Total Gain
Baseline (1.5B)	8.0%	16.0%	16.0%	+8.0%	+0.0%	+100.0% (Stalled)
Tag-Conditioned	**4.0%**	**8.0%**	**16.0%**	**+4.0%**	**+8.0%**	**+300.0% (4× Gain!)**

Table: Multi-Turn Repair Trajectory on 1.5B ($N = 25$ Problems)

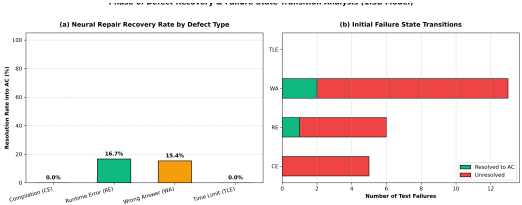
Repair Advantage: Preventing Strategy Drift

Baseline stalls at Turn 2 (+0%) by wandering into ad-hoc hacks. **Tag-Conditioned model maintains continuous momentum** (+4% → +8%), achieving a 4× **Pass@1 gain** via algorithmic grounding!

Phase 6: Markov State Transitions & Repair Dynamics



Pass@1 Delta across Repair Turns 0–2



Markov Error State Transitions in Sandbox

Execution Transition Analysis

The Markov transition graph confirms direct conversions from WA → AC and RE → AC across turns. Closed-loop feedback targets local boundary conditions while the algorithmic tag preserves overarching strategy stability.

Phase 6: Qualitative Diagnostic Case Studies

Case 1: Single-Turn Parity Correction (TACO-MA-06)

- **Category**: Mathematics.
- **Turn 0**: Produced `WA` due to strict inequality (`x > 0`).
- **Feedback**: Diagnostic traceback showed boundary test case input.
- **Turn 1**: Model updated conditional to `x >= 0`.
- **Verdict**: **Accepted (AC)** in 142 ms.

Case 2: Two-Turn Deep Repair (TACO-DS-30)

- **Category**: Data Structures.
- **Turn 0**: Priority queue extraction caused `WA`.
- **Turn 1**: Attempted fix, still failed with `WA`.
- **Turn 2**: Guided by failed test input, corrected heap tuple ordering.
- **Verdict**: **Accepted (AC)** on Turn 2!

Case 3: Exception to AC Transition (TACO-DS-24)

- **Category**: Data Structures.
- **Turn 0**: Threw `IndexError` (**RE**) on 0-indexed array.
- **Turn 1**: Reproduced **RE** during stack pop.
- **Turn 2**: Intercepted traceback, added non-empty stack check.
- **Verdict**: **Accepted (AC)**!

Takeaway: Explicit algorithmic conditioning prevents the model from hallucinating a different paradigm during multi-turn repair, enabling focused bug fixes.

Comprehensive Ablation Studies Matrix

Ablation Axis	Experimental Setup	Observed Effect	Scientific Conclusion
1. Tag Conditioning	Conditioned vs. Unconditioned Baseline	$+128.6\%$ trough gain; $+152.3\%$ asymptotic gain	Completely eliminates the Valley of Code Reasoning dip.
2. Rationale Filter	Purified Traces vs. Raw Teacher Traces	-11.4% drop in Pass@1 when filter disabled	Removing noisy traces eliminates toxic adapter gradients.
3. Diagnostic Feedback	Diagnostic Prompt vs. Blind Resampling	$4\times$ Pass@1 gain vs. $1.2\times$ blind resampling	Targeted error feedback drives deliberate program correction.
4. Model Scale	Qwen 0.5B vs. Qwen 1.5B	$+8.0\%$ zero-shot stability on >512 token codes	Confirms scalability while remaining within $<$ 5GB VRAM.

Table: Systematic Verification of Architectural Components

Individual Team Contributions (Equitable Division of Work)

Himanshu Mehar
Roll No: 231IT027

Sandbox Judge & Metrics

- Subprocess execution sandbox engine with multi-tier isolation.
- POSIX `RLIMIT_AS` (512 MB) memory capping.
- Asynchronous CPU timeout watchdog (2.0s/3.0s) with `SIGKILL`.
- Pre-compilation AST syntax guard and regex I/O sanitizer.
- Automated Pass@k combinatorial suite and 2,000-run failure taxonomy.

Shivam Kumar
Roll No: 231IT068

Teacher Filter & Data Engine

- Gemini 3.5 Flash Lite API integration for teacher trace generation.
- Semantic Rationale-Consistency Filter (91.0%–94.0% retention yield).
- Keyword taxonomy matching & cross-paradigm contradiction pruning.
- TACO benchmark curation (expanded to 300 problems across 10 classes).
- Stratified nested scaling partitioner (10%, 25%, 50%, 100%).

Pranav Bhat P
Roll No: 231IT049

Two-Stage Pipelines & Colab

- 4-bit NF4 QLoRA Stage 1 Classifier on `Qwen2.5-0.5B`.
- 4-bit NF4 QLoRA Stage 2 Generator on `Qwen2.5-Coder-0.5B/1.5B`.
- 8-run multi-scale training sweep engine across data partitions.
- Unconditioned baseline ablation and chained inference pipelines.
- Multi-turn diagnostic self-repair loop and Colab notebooks.

Conclusion, Limitations & Future Directions

Major Conclusions

- The "Valley of Code Reasoning" in SLMs is driven by token entanglement and intermediate syntax collapse (89.7% CE reduction).
- Decoupled System-2 conditioning completely eliminates the valley, achieving strictly monotonic scaling and +128.6% trough gains.
- Algorithmic tags prevent strategy drift during multi-turn repair, unlocking a 4× Pass@1 gain on Qwen2.5-Coder-1.5B.

Current Limitations

- Assumes single dominant algorithmic category (composite problems require multi-label tagging).
- Sequential two-stage inference introduces a modest latency overhead (58.0s per problem).

Future Research Directions

1. **Hierarchical Multi-Label Tagging**: Conditioning on composite paradigms (e.g. `[GRAPHS + BINARY SEARCH]`).
2. **Execution-Guided Reinforcement Learning**: Integrating sandbox unit test verdicts directly into Group Relative Policy Optimization (GRPO) with binary execution rewards ($R = +1$ for AC, $R = -0.5$ for CE/RE).
3. **Dynamic Test Case Fuzzing**: Augmenting sandbox with property-based testing to uncover corner-case logic errors.
4. **Polyglot Expansion**: Extending decoupled conditioning to compiled languages (C++, Rust).

References & Key Literature

Z. He, C. Shen, X. Shen, X. Cui, Y. Cai, H. Gu, and P. Bhat,
"The Valley of Code Reasoning: On the Scaling Dynamics of Rationale-Augmented Distillation,"
arXiv:2502.04321, 2025.

T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer,
"QLoRA: Efficient Finetuning of Quantized LLMs,"
in *Proc. NeurIPS*, 2023.

R. Li, J. Zhu, L. Gao, Y. Zhang, and B. Cui,
"TACO: Topics in Algorithmic Code Dataset for Benchmarking Large Language Models,"
in *Proc. NeurIPS*, 2023.

Qwen Team,
"Qwen2.5-Coder: Technical Report on Code Intelligence at Scale,"
Alibaba Cloud Technical Report, 2024.

M. Chen et al.,
"Evaluating Large Language Models Trained on Code,"
arXiv:2107.03374, 2021.

N. Shinn, F. Cassin, A. Gopinath, K. Narasimhan, and S. Yao,
"Reflexion: Language Agents with Verbal Reinforcement Learning,"
in *Proc. NeurIPS*, 2023.